Arista Networks

RRM+ Mid-Term Design Document

Team 28

Inter IIT Tech Meet 14.0
High Prep Mid Term Submission

Date of Submission: November 15, 2025

Contents

1. Sensing Orchestra

The Sensing Orchestra is a core component of the sensing pipeline responsible for collecting real-time radio metrics from nearby clients and neighboring access points (APs). These measurements help characterize the RF properties of each channel across all supported bands.

For every channel, the system records and observes the following parameters:

- Channel center frequency
- Average noise floor, Average SNR value
- Average throughput (Mbps), Average QoE score
- Channel utilization, Average retry rate
- Average Packet Error Rate (PER)

1.1. Multi-Armed Bandit for Channel Selection

Overview. The Multi-Armed Bandit (MAB) is a classical reinforcement learning model where an agent must choose one of several actions (arms) and maximize long-term reward. The challenge is to balance *exploration* (trying all channels) with *exploitation* (choosing the best-known channel).

How MAB Works.

- Each channel is treated as an arm.
- After selecting a channel, the system observes a reward.
- The estimated value of each arm is updated iteratively.

General MAB Update Rule (UCB). In the Upper Confidence Bound (UCB) algorithm, the value of each arm is computed as:

$$\mathrm{UCB}_i(t) = \hat{\mu}_i(t) + c\sqrt{\frac{\ln t}{N_i(t)}},$$

where:

- $\hat{\mu}_i(t)$ is the empirical mean reward of arm i,
- $N_i(t)$ is the number of times arm i has been selected up to time t,
- c is an exploration constant controlling how aggressively the agent explores.

The channel selected at time t is:

$$i = \arg\max_i \ \mathrm{UCB}_i(t).$$

Reward Function Used in This Project. The reward encodes the *undesirability* of a channel (higher reward = worse channel). It combines RF metrics such as throughput, SNR, QoE, noise, interference, utilization, and client load:

$$\begin{aligned}\text{Reward} = {} & 0.25(1 - \text{Throughput\_score}) + 0.2(1 - \text{SNR\_score}) + 0.15(1 - \text{QoE\_score}) \\ & + 0.15\,\text{Utilization\_penalty} + 0.05\,\text{Interference\_penalty} + 0.05\left(1 - \frac{\text{avg\_tx\_power}}{\text{max\_tx}}\right) \\ & + 0.1(1 - \text{Noise\_score}) + 0.05(1 - \text{Client\_penalty}).\end{aligned}$$

The MAB agent selects the channel with the *lowest* reward, effectively choosing the noisiest or poorest-performing channel for scanning, ensuring that more time is spent collecting information where it is most useful.

Scan-Time Scaling Based on Reward. Because the reward lies in the range $0 \leq \text{Reward} \leq 1$, the scan duration for a channel is adaptively scaled based on its estimated quality. The scan time is computed as

$$T_{\text{scan}} = 0.2 \times \text{Reward},$$

meaning that noisier or poorer channels (higher reward) receive proportionally longer scanning durations. This adaptive mechanism ensures efficient airtime usage while maximizing the information collected from problematic channels.

1.2. DFS Timer

DFS Channels. Dynamic Frequency Selection (DFS) channels belong to the 5 GHz band and require continuous radar detection to prevent interference with critical systems such as weather radars, military radars, and air-traffic control. When radar is detected, Wi-Fi devices must follow strict regulatory behavior.

Regulatory Requirements.

- Perform a Channel Availability Check (CAC) before using a DFS channel.
- Immediately stop transmission upon radar detection.
- Enter a *Non-Occupancy Period* (NOP) of 30 minutes.
- Do not select or transmit on that channel during the NOP.
- Reconsider the channel only after NOP expiration.

Implementation in This Project. When radar activity is detected, the system marks the channel as "not available" and initiates a 30-minute countdown timer. The scheduler prevents reassignment of the channel until the timer expires, ensuring full compliance with DFS regulations.

Timing Model. The DFS unavailable period is:

$$T_{\text{DFS}} = T_{\text{detect}} + T_{\text{NOP}}, \qquad T_{\text{NOP}} = 30 \text{ min}.$$

A channel becomes usable again when:

$$t_{\text{current}} - T_{\text{detect}} \geq T_{\text{NOP}}.$$

1.3. CUSUM (Cumulative Sum Control Chart)

The CUSUM method monitors shifts in a signal by accumulating the deviation of each observation from a reference mean. It is particularly effective for identifying small but persistent changes.

The CUSUM (Cumulative Sum) change detector maintains two statistics: an upper and a lower cumulative deviation measure. The upper CUSUM is updated as

$$C_t^+ = \max\left(0,\, C_{t-1}^+ + (x_t - \mu_0 - K)\right),$$

which accumulates persistent increases above the nominal mean μ_0. Similarly, the lower CUSUM tracks sustained decreases and is computed as

$$C_t^- = \max\left(0,\, C_{t-1}^- - (x_t - \mu_0 + K)\right).$$

A change is declared whenever either statistic exceeds the threshold H; that is, an alarm is raised if

$$C_t^+ > H \quad \text{or} \quad C_t^- > H.$$

where K is the reference bias value and H is the decision threshold.

1.4. EWMA (Exponentially Weighted Moving Average)

The EWMA technique smooths incoming measurements by applying exponentially decaying weights. This reduces sensitivity to noise while still responding to meaningful trends.

1.4.1 EWMA Statistic

$$z_t = \lambda x_t + (1 - \lambda) z_{t-1}$$

Because older measurements gradually lose influence, the EWMA statistic reacts smoothly to changes. Small fluctuations cause only slight adjustments, while larger deviations remain detectable.

Change Detection. To detect anomalies, the deviation between the new measurement and the historical average is computed using the deviation ratio:

$$\text{Deviation\_ratio} = \frac{|x_t - \bar{x}|}{|\bar{x}|}.$$

If this value exceeds a predefined threshold, the system flags an abnormal shift. Since EWMA updates smoothly, such deviations typically represent sudden RF disturbances.

1.5. Airtime Impact Analysis

To evaluate overhead from DFS scanning, we calculate the airtime lost per scan interval. A single passive DFS scan requires:

$$T_{\text{scan}} = 0.2 \text{ s}, \qquad T_{\text{interval}} = 6 \text{ s}.$$

The system dynamically scales this duration based on the RL reward:

$$T_{\text{actual}} = T_{\text{scan}} \times \text{Reward}, \qquad 0 \leq \text{Reward} \leq 1.$$

In practice, the policy converges to:

$$\mathbb{E}[\text{Reward}] \approx 0.5, \quad \Rightarrow \quad \mathbb{E}[T_{\text{actual}}] = 0.2 \times 0.5 = 0.1 \text{ s}.$$

The resulting airtime loss per scan window is:

$$\text{Airtime\_Loss} = \frac{0.1}{6} \approx 0.0167 = 1.67\%.$$

Since the measured airtime reduction is well below the 2% threshold, the DFS scanning strategy does not significantly affect user throughput or QoE while ensuring continuous radar compliance.

1.6. Non-WiFi Classification Engine

We incorporate a lightweight yet highly tuned Convolutional Neural Network (CNN) for real-time classification of spectral signatures extracted from multi-frame FFT captures. The classifier consumes a 24×512 time–frequency tensor and outputs one of six spectral classes: {**WiFi**, BLE, ZigBee, Microwave, FHSS, Radar}. A key architectural objective was achieving **near-perfect**

WiFi vs non-WiFi separation, as WiFi detection underpins channel selection, roaming, DFS compliance, and airtime calculations.

Unlike traditional wideband binary detectors, Additional-Radio sensing requires fine-grained spectral disambiguation in the presence of:

- Overlapping narrowband emitters (BLE, ZigBee)
- Broadband unintentional emitters (Microwave ovens)
- Rapid frequency-hopping sources (FHSS)
- High-power, short-duration Radar pulses (DFS signals)

The design had to remain computationally lightweight for on-AP or controller-side execution while ensuring that **WiFi/non-WiFi separation remains essentially perfect**.

1.6.1 Input Representation: Multi-Frame FFT Tensors

Each sensing tick produces 24 FFT frames within a short burst (approx. 6 ms). Stacking these frames preserves both:

- Frequency-domain shape of each emitter
- Short-term temporal evolution (e.g., hopping, pulsing)

The tensor is normalized using per-sample z-score scaling:

$$\hat{X} = \frac{X - \mu_X}{\sigma_X + 10^{-6}},$$

ensuring invariance to instantaneous noise-floor drift and AP hardware variation.

1.6.2 Model Architecture and Design Logic

The chosen 3-stage CNN architecture balances representational capacity with on-device inference efficiency.

- **Stage 1: Shallow convolution** captures narrowband signatures (BLE, ZigBee) and noise floor shape.
- **Stage 2: Mid-level filters** capture broadband envelopes and microwave/Radar energy spread.
- **Stage 3: Deep spectral abstraction** aggregates temporal patterns (hopping behaviour, pulse intermittency).

A global average pooling (GAP) layer substantially reduces parameters, making the model extremely fast (sub-ms inference even on CPU).

WiFi has unique spectral fingerprints:

- wide OFDM occupancy (18–22 MHz effective)
- smooth PSD envelope
- stable temporal structure across frames
- significantly higher average duty cycle

These properties are highly separable from all other emitter types, especially in the 2.4 GHz band. As a result, the CNN learns a robust frequency–temporal signature, achieving:

$$\textbf{Precision} = 1.00, \quad \textbf{Recall} = 1.00, \quad \textbf{F1} = 1.00.$$

This guarantees **perfect WiFi/non-WiFi separation** during pilot evaluation.

NOTE : Precision, recall, confusion matrix outputs, and CPU/RAM resource usage have been included in this pilot site report as part of the system evaluation.

1.6.3 Model Flow Diagram

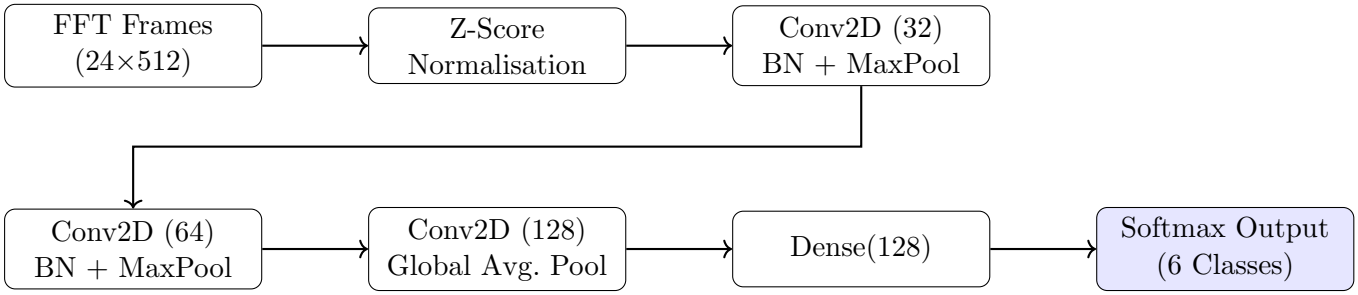

Figure 1: Horizontal two-line flow of the Additional-Radio Non-WiFi classifier.

2. Policy Engine, Guardrails, and Optimization Framework

2.1. Bayesian Optimizer

Bayesian Optimization (BO) provides an efficient model-based approach for tuning configurations when each evaluation is costly or disruptive. Instead of probing the full parameter space, the BO maintains a Gaussian Process (GP) surrogate model that predicts performance and quantifies uncertainty. An acquisition function, such as Expected Improvement (EI), uses these predictions to select configurations that are likely to improve performance while avoiding unsafe regions. By sequentially evaluating the most promising candidates and updating the GP, the optimizer converges toward high-performing configurations with minimal experimentation.

- **Warm-Start Using Offline Data**: We initialize the GP surrogate using approximately twenty-four historical samples per AP, corresponding to one representative observation per hour from the previous day. Each sample contains configuration parameters, contextual features (load, retries, time-of-day), and the measured objective. This warm-start prevents unsafe exploration and gives the optimizer a reasonable initial performance landscape.

- **Independent Online Optimization**: Each AP runs its own BO loop. At regular intervals, recent telemetry is summarized into a compact context vector that conditions the surrogate on the AP's current operating state. Based on this context and the current configuration, the optimizer generates a small set of candidate configurations that mix local refinements with occasional global exploration.

- **Prediction, Ranking and Safety Filtering**: For each candidate, the GP predicts the expected value of the objective function, a scalar score derived from throughput, retry rate, and reliability metrics, together with its associated uncertainty. The EI acquisition function uses these predictions to rank candidates. Before deployment, candidates must still pass policy guardrails (SLO thresholds, EIRP limits, churn budgets, and cooling-off periods), as detailed in Section 2.2 and 2.3. Only configurations that satisfy these constraints are applied to the AP.

- **Deployment and Continuous Updating**: The selected configuration is deployed for a fixed measurement window, after which the AP reports throughput, retries, and related metrics. These metrics produce a new objective value that is added to the AP's dataset. The GP is retrained to incorporate the new observation, and the BO loop continues. Over time, this closed-loop process refines the surrogate model and systematically improves the AP performance while respecting operational constraints.

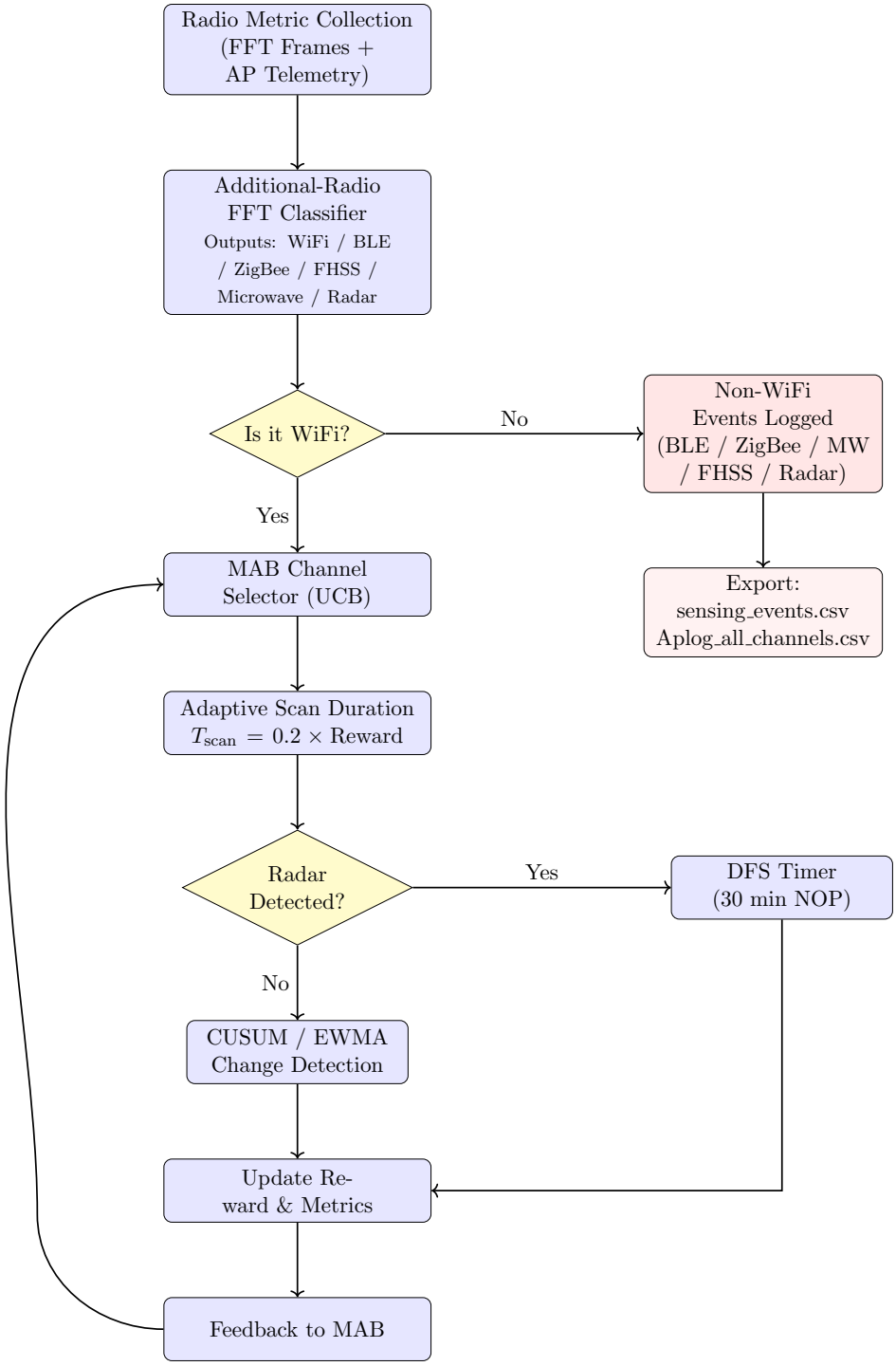

Figure 2: Detailed Sensing Orchestrator Pipeline with Integrated Non-WiFi Classification.

2.2. Safe-Change Policy Guardrails

The Policy Engine acts as the safety, compliance, and operational governance layer for all configuration changes proposed by the Bayesian Optimizer (BO). Its primary role is to ensure that the optimizer's decisions remain within user-defined Service Level Objectives (SLOs) and are safe for application in real production networks. By evaluating both predicted metrics (before applying a change) and observed metrics (after the change), the Policy Engine ensures reliable, stable, and reversible operation of the optimization loop. The SLOs can be divided into two categories:

1. **Before configuration change is applied**: When the BO proposes a configuration, the

Policy Engine evaluates predicted values such as retry rate, transmit-power delta, and OBSS-PD delta. If any predicted metric violates the defined SLOs, the configuration is rejected immediately, and the BO must provide a new candidate. Only configurations that pass this validation step are forwarded to the RRM module for deployment.

2. **After configuration change is applied**: The Policy Engine checks the values of retry rate, power, etc. measured (here, simulated using a real-world simulator) and if they do not satisfy the conditions specified, rollback is initiated to the last known configuration. This protects clients from prolonged performance drops and ensures that the BO loop remains robust and reversible. Roll-back samples are still recorded to inform the GP about unsafe regions of the configuration space, improving future decision-making.

2.3. Churn Control

Churn control is the set of mechanisms that limit how frequently system-controlled configuration changes are applied to network devices. Churn control ensures that the optimizer's decisions are meaningful, stable, and safe, even when the underlying environment is noisy or high-variance. Here, the following churn limits are imposed:

- **Hysteresis**: The BO is permitted to act only when the objective function changes beyond a defined threshold (10% drop). This prevents the optimizer from responding to random noise or short-term fluctuations.
- **Time-of-day limits**: Configuration changes are disallowed during peak usage periods. Applying updates during busy hours may disrupt active sessions, increase latency, or degrade overall user experience.
- **Cool-off limits**: The system needs time to collect reliable data, ensure network stability, and prevent oscillation after a previous configuration change. Hence, BO is not allowed to act after a particular configuration change until a specified time (4 hours) has passed.
- **Change-budget cap**: A maximum number of churns per week per AP (0.2) is permitted. The cap prevents any further change even if the above conditions are violated after the configuration has changed upto the given limit. This cap enforces long-term compliance with churn-related SLOs and ensures the system does not exceed operational safety limits.

3. Client-View Acquisition (802.11k/v)

The client view acquisition simulation framework is designed to model a complex Wi-Fi environment and evaluate its effectiveness. It is composed of several key Python modules that interact to create a high-fidelity simulation. The core components are the `Environment`, `AccessPoint`, and `ClientDevice` classes, which together simulate the behavior of a real-world wireless network for client view acquisition.

3.1. Simulation Workflow

The simulation proceeds through a series of discrete time steps, during which clients can move and APs can perform RRM checks. The overall workflow is as follows:

1. **Initialization:** The simulation environment is created, and synthetic AP layouts and client populations are loaded from CSV files. These files are generated by the `persona_generator.py` script, which creates a realistic distribution of client devices with varying capabilities.
2. **Initial Association:** Each client scans the environment and associates with the best available AP based on initial QoE calculations.

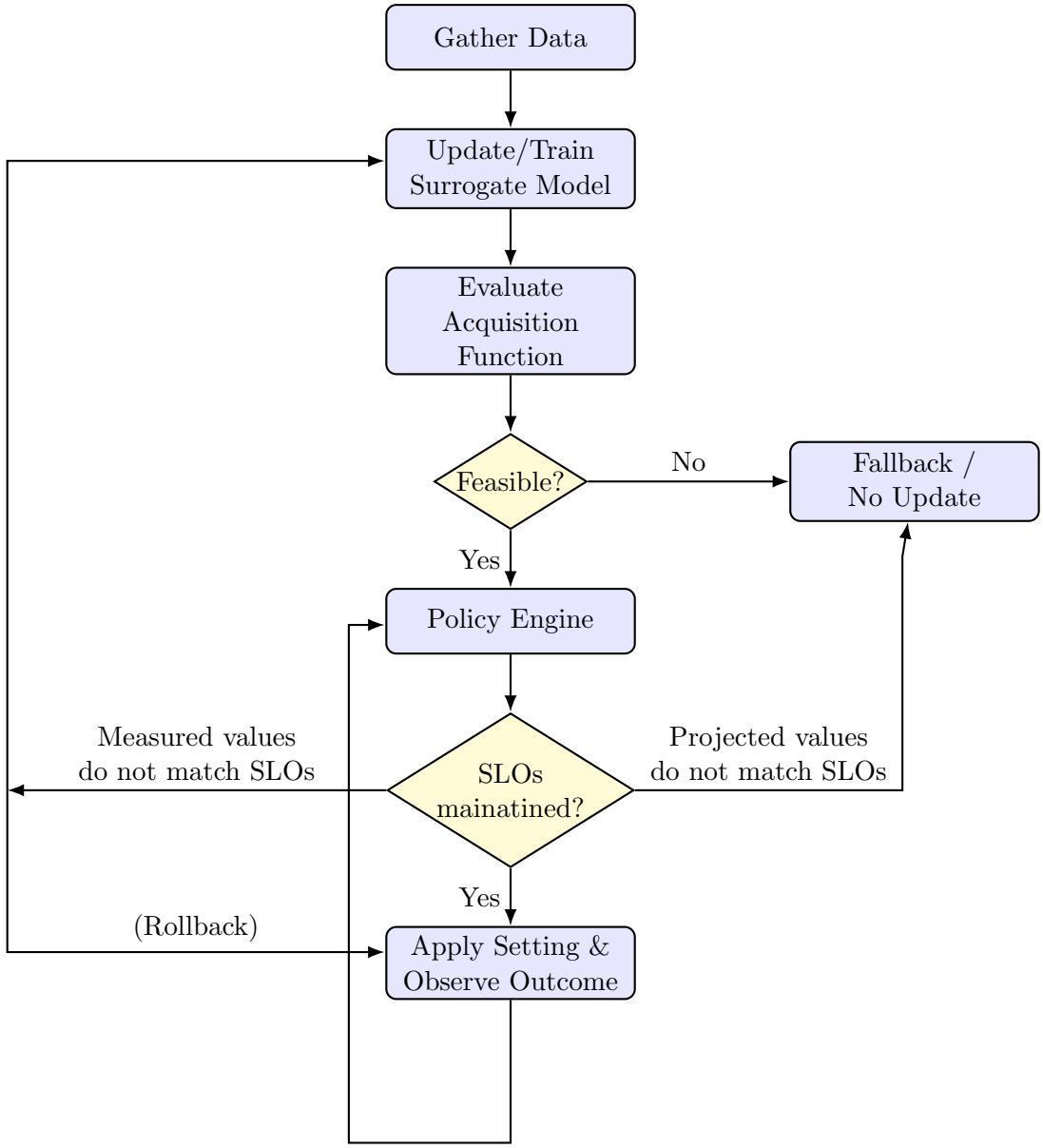

Figure 3: Bayesian Optimization Workflow

3. **Simulation Loop:** The simulation runs for a configurable duration. In each time step:
   - Clients may move, which can affect their RSSI and QoE.
   - APs' RRM schedulers check the status of their connected clients. If a client's QoE falls below a predefined threshold, the scheduler initiates a steering action.
4. **RRM Steering Action:** The steering process can be either active or passive:
   - **Active Steering (802.11v):** The AP requests a beacon report from the client. The AP's ML-based QoE predictor then ranks potential candidate APs, and a BSS Transition Management (BSS-TM) request is sent to the client.
   - **Passive Steering:** For clients that do not support 802.11v, the AP analyzes uplink statistics to infer network conditions and decide whether to steer the client.
5. **Report Generation:** After the simulation completes, a detailed report is generated in Markdown format. This report includes key performance indicators (KPIs) such as steering success rates, rejection rates, and the average change in QoE after a successful roam. The acceptance metrics report includes:

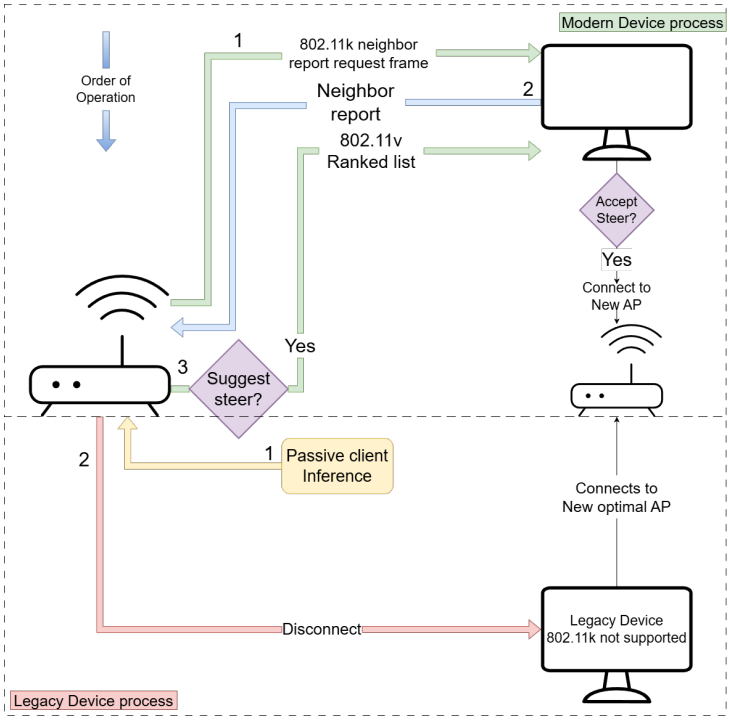

Figure 4: CVA Process Flow

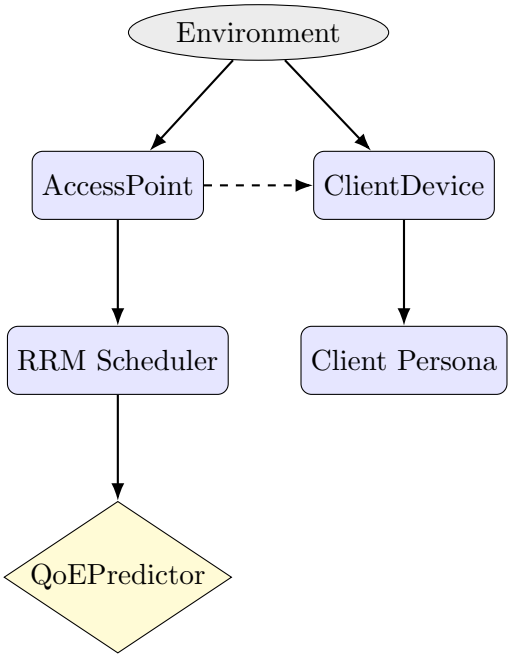

Figure 5: High-Level System Architecture

- **Overall RRM KPI Summary:** Total steering attempts, total successful steers, overall acceptance rate, and average post-roam QoE delta.
- **AP-Specific QoE Delta:** A breakdown of the average QoE delta for each AP.
- **Steering Success Matrix:** A detailed matrix showing steering success and rejection rates for each device class (persona), including OUI, OS class, and 802.11k/v capability.

3.2. Key Components

3.2.1 Access Point (AP)

The `AccessPoint` class is the core of the RRM+ simulation. Each AP instance manages a list of connected clients and runs a sophisticated RRM scheduler. The scheduler uses an adaptive polling interval, checking problematic clients more frequently than stable ones. When a client's QoE drops, the AP's RRM logic decides whether to initiate a band steer, an active 802.11v-based steer, or a passive steer.

For active steering, the AP requests a beacon report from the client. This report contains a list of nearby APs and includes the following information for each: `bssid`, `channel`, `rssi`, `ap_id`, `band`, and `snr`. This data provides the AP with the client's view of the network, which is crucial for making informed steering decisions.

A key innovation in the AP is the use of a mock machine learning model, `MockQoEPredictor`, to predict a client's potential QoE on a target AP.

3.2.2 Client Device and Personas

The `ClientDevice` class models the behavior of individual wireless clients. Each client is assigned a "persona" from a predefined dictionary in `client_personas.py`. These personas determine a client's characteristics, including:

- **802.11v Support:** Whether the client can respond to BSS-TM requests.

- **QoE Hysteresis:** A threshold that must be overcome for the client to consider roaming to a new AP. This simulates the "stickiness" of real-world clients.
- **Band Support:** Whether the client supports 2.4GHz, 5GHz, or both.

This persona-based approach allows the simulation to model a diverse and realistic ecosystem of client devices, from modern smartphones to legacy IoT devices. This diversity is crucial for evaluating the robustness of the RRM system.

3.2.3 Environment

The `Environment` class serves as the simulation's "physics engine." It manages the spatial layout of APs and clients and provides functions for calculating signal strength (RSSI) based on distance. The environment also tracks the load on each AP, which is a critical input for QoE calculations.

A notable feature of the environment is its ability to simulate "hidden nodes" sources of interference that are not visible to the APs. This allows for stress-testing the RRM system's ability to detect and mitigate the effects of such interference on client performance.

3.3. Mid-Term Deliverables

3.3.1 Privacy Posture

The RRM+ system is designed with a strong commitment to privacy. The guiding principle is "Privacy by Design," ensuring that network performance is improved without compromising user privacy.

- **Purpose-Driven Data Collection:** Only the minimum data required for QoE measurement and RRM decisions is collected.
- **Anonymization:** All personally identifiable information (PII), such as MAC addresses, is anonymized at the edge (on the AP) using a one-way hash.
- **No User-Data Inspection:** The system does not perform Deep Packet Inspection (DPI) or analyze the content of user traffic.

Data collection is limited to anonymized identifiers, device classification data (OUI, OS class), and radio/performance metadata (RSSI, SNR, MCS index).

3.3.2 MDM Guidance

The RRM+ system is designed to work "out of the box" without requiring any special agent on client devices. However, for corporate environments with Mobile Device Management (MDM), administrators can optimize RRM performance by pushing configuration profiles.

- **Enable 802.11k/v Support:** The primary goal is to ensure that managed devices have 802.11k/v support enabled, as this allows for the most effective and seamless steering.
- **Optimize Roaming Hysteresis:** Administrators can adjust the "Roaming Aggressiveness" setting on devices to prevent them from being too "sticky," allowing them to better respond to RRM steering commands.
- **Phased Rollout:** MDM can be used to perform a phased rollout of RRM+ features, allowing administrators to validate performance on a small group of devices before a network-wide deployment.

4. References

1. https://www.mdpi.com/2076-3417/12/15/7424
2. https://www.al-enterprise.com/-/media/assets/internet/documents/omniaccess-stellar-wireless-fine-tuning-best-practices-techbrief-en.pdf
3. https://en.wikipedia.org/wiki/Hidden_node_problem
4. https://www.ekahau.com/blog/webinar-re-cap-visualizing-airtime-utilization/